LAPORAN PRAKTIKUM WEB SERVICE SESI 08

API Gateway Sederhana

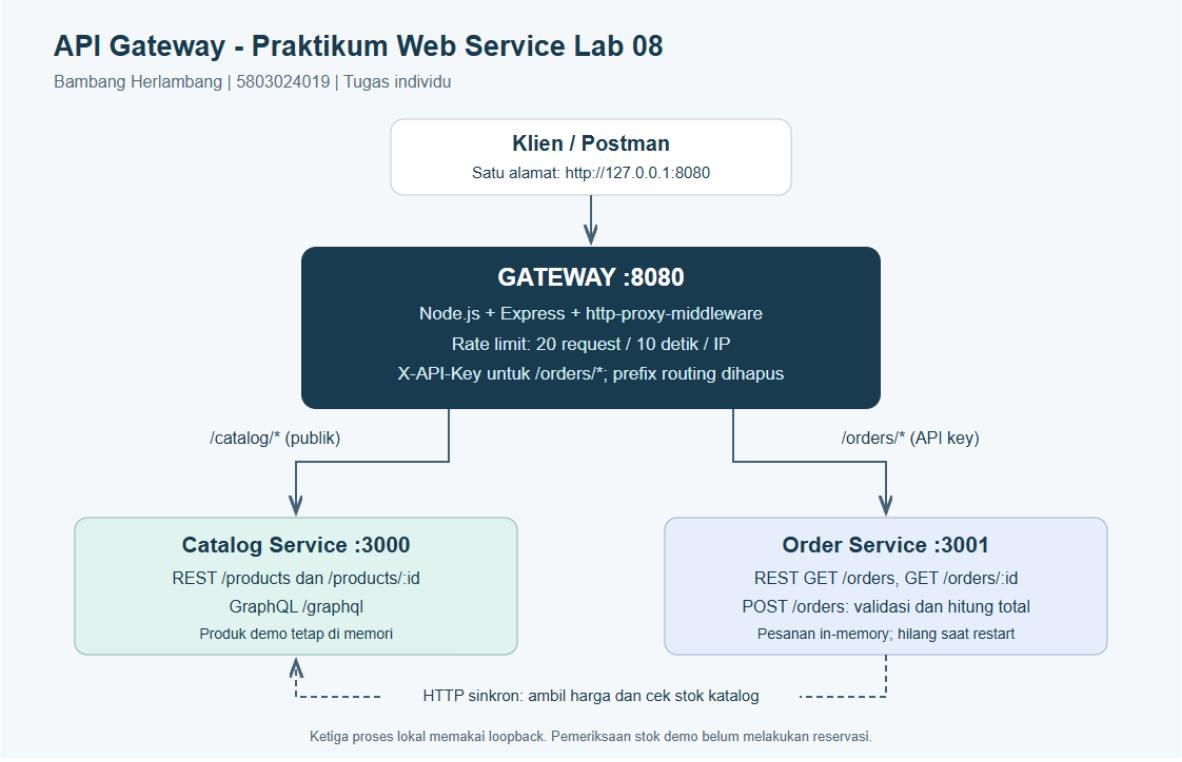
Node.js dan Express

Nama	Bambang Herlambang
NIM	5803024019
Pengerjaan	Individu
Tanggal	21 Oktober 2026
Repository	academicbambang/prakwebservice-lab08-api-gateway

1. Tujuan dan ruang lingkup

Praktikum ini membangun satu pintu masuk untuk dua service demo: Catalog Service dan Order Service. Gateway meneruskan request berdasarkan path, membatasi request menjadi 20 per 10 detik per alamat IP, serta memeriksa API key pada jalur pesanan. Implementasi mengikuti Opsi A Lab 08 dengan Node.js, Express, `http-proxy-middleware`, dan `express-rate-limit`.

Pembuktian awal meliputi akses kedua service melalui port 8080, penolakan akses tanpa key, dan perubahan respons menjadi `429` ketika kuota terlampaui; deployment publik dilaporkan pada Bagian 8. Tugas dikerjakan secara individu; kedua service dipakai sebagai konteks demonstrasi gateway. Materi Web Service menjadi acuan keamanan API key, sedangkan materi Software Testing menjadi acuan pengujian batas dan skenario negatif.



Gambar 1. Arsitektur implementasi lokal. Order Service mengambil harga dan memeriksa stok melalui panggilan HTTP sinkron ke Catalog Service.

2. Arsitektur dan implementasi

2.1 Routing dua service

Klien cukup menggunakan `http://127.0.0.1:8080`. Prefix `/catalog` atau `/orders` dihapus sebelum request diteruskan ke port 3000 atau 3001. Query string dan body tetap diteruskan.

Method	Path pada gateway	Path pada service
GET	`/catalog/products`	Catalog `/products`
GET	`/catalog/products/:id`	Catalog `/products/:id`
POST	`/catalog/graphql`	Catalog `/graphql`
GET	`/orders/orders`	Order `/orders`
GET	`/orders/orders/:id`	Order `/orders/:id`
POST	`/orders/orders`	Order `/orders`

Tabel 1. Pemetaan endpoint melalui gateway.

Path `/orders/orders` disengaja: bagian pertama merupakan namespace gateway dan bagian kedua merupakan resource pada Order Service. Endpoint `GET /health` tersedia pada ketiga proses.

2.2 Rate limiting, API key, dan penerusan request

Middleware rate limit dipasang sebelum routing API. Endpoint API, termasuk `/health`, respons `401`, dan route tidak dikenal, berbagi kuota untuk IP yang sama. Request ke-21 dalam window yang sama mendapat `429`. Penghitung memakai memori satu proses. Endpoint readiness `/readyz` untuk deployment diletakkan di luar limiter.

Katalog bersifat publik, sedangkan jalur pesanan memerlukan header `X-API-Key`. Key yang tidak ada atau tidak cocok ditolak dengan `401 Unauthorized`. Pemeriksaan menggunakan digest SHA-256 dan `timingSafeEqual`; digest server juga dapat diberikan melalui `API_KEY_SHA256`. Key asli dihapus dari request yang diteruskan ke upstream.

Gateway tidak memasang `express.json()` sebelum proxy sehingga body request tetap utuh. Parsing dilakukan pada service tujuan. Header `Location` relatif dari upstream ditulis ulang dengan prefix gateway agar tautan detail pesanan tetap melewati gateway. Upstream yang tidak tersedia menghasilkan respons `502` yang terkendali.

2.3 Konfigurasi dan data demo

Konfigurasi gateway berada di `gateway/.env`; contoh tanpa secret tersedia dalam `gateway/.env .example`. File `.env` diabaikan oleh Git.

Variabel	Pengaturan lokal / fungsi
`HOST`, `PORT`	`127.0.0.1`, `8080`
`CATALOG_SERVICE_URL`	`http://127.0.0.1:3000`
`ORDER_SERVICE_URL`	`http://127.0.0.1:3001`
`API_KEY`, `API_KEY_SHA256`	Key acak lokal atau digest alternatif
`TRUST_PROXY`	`0` untuk koneksi lokal langsung
`CATALOG_PORT`, `ORDER_PORT`	Port backend `3000` dan `3001` pada stack

Tabel 2. Konfigurasi utama. Port backend terpisah dari variabel `PORT`.

Catalog Service menyediakan REST dan GraphQL. Order Service mengambil harga produk dari katalog, memvalidasi kuantitas, serta menghitung total pesanan. Data disimpan dalam memori; stok diperiksa tanpa reservasi atau pengurangan.

3. Prosedur dan rancangan pengujian

3.1 Reproduksi lingkungan lokal

Gunakan Node.js 22 atau lebih baru. Dari root repository, jalankan:

```
npm install
npm run setup
npm start
```

Setup membuat key acak pada `gateway/.env`. Import collection dan environment di `docs/postman` ke Postman, pilih environment lokal, kemudian isi variabel `apiKey` dengan key asli untuk klien. Jangan mengirim digest server sebagai key klien.

Untuk menjalankan proses secara terpisah, gunakan tiga terminal:

```
node catalog-service/server.js
node order-service/server.js
node gateway/gateway.js
```

Perintah `node gateway.js` juga dapat dijalankan dari folder `gateway` setelah kedua backend aktif.

3.2 Strategi dan skenario

Suite memakai `node:test`. Unit test memeriksa logika yang terisolasi, integration test memeriksa perilaku HTTP, dan pengujian alur menyeluruh menghubungkan gateway dengan kedua service nyata. Boundary Value Analysis (BVA) diterapkan pada batas kuota 20/21 serta batas kuantitas pesanan; skenario negatif diperiksa dengan assertion otomatis.

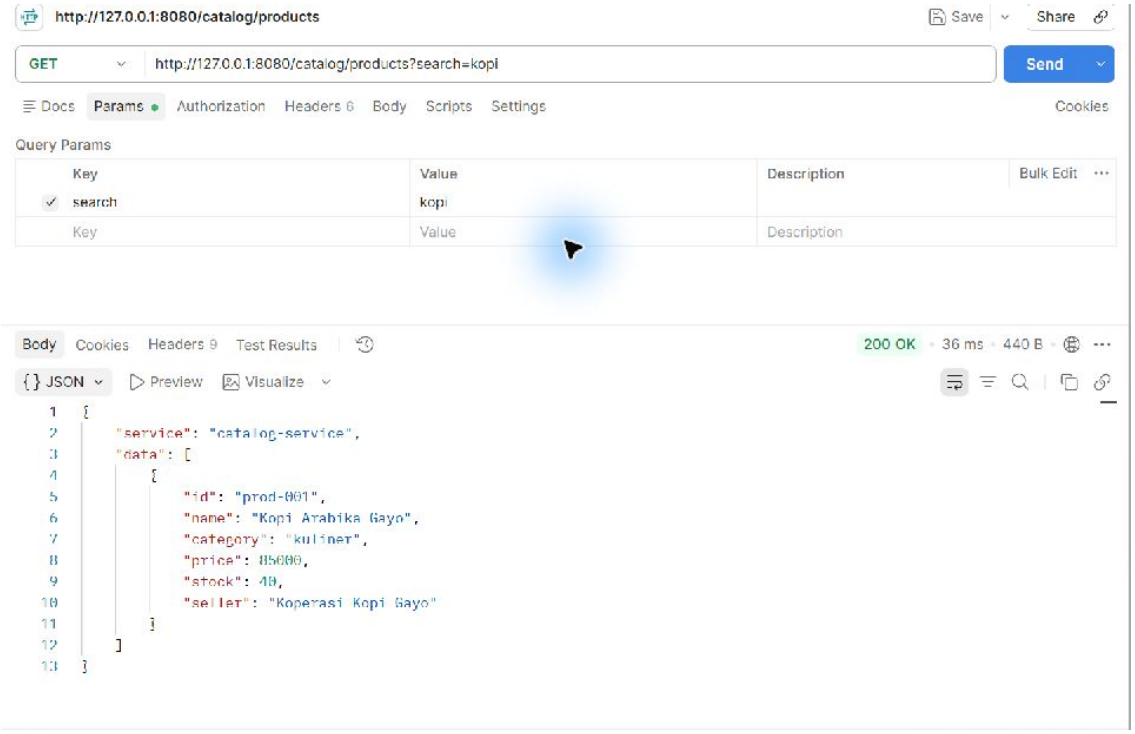
ID	Skenario	Hasil yang diharapkan
GW-01	Katalog melalui gateway	`200`, data katalog
GW-02	Pesanan dengan key benar	`200`, data Order Service
GW-03	Key tidak ada atau salah	`401`, akses ditolak
GW-04	Penghapusan prefix dan query	Path serta query sesuai upstream
GW-05	POST JSON GraphQL dan pesanan	Body utuh, respons selesai
GW-06	Route tidak terdaftar	`404`
GW-07	30 request pada window bersih	20 respons `200`, lalu 10 `429`
GW-08	Window rate limit berakhir	Kuota kembali tersedia
GW-09	Input pesanan tidak valid	`400`, pesanan tidak dibuat
GW-10	Upstream tidak tersedia	`502`, request tidak menggantung

Tabel 3. Skenario fungsional, batas, dan negatif.

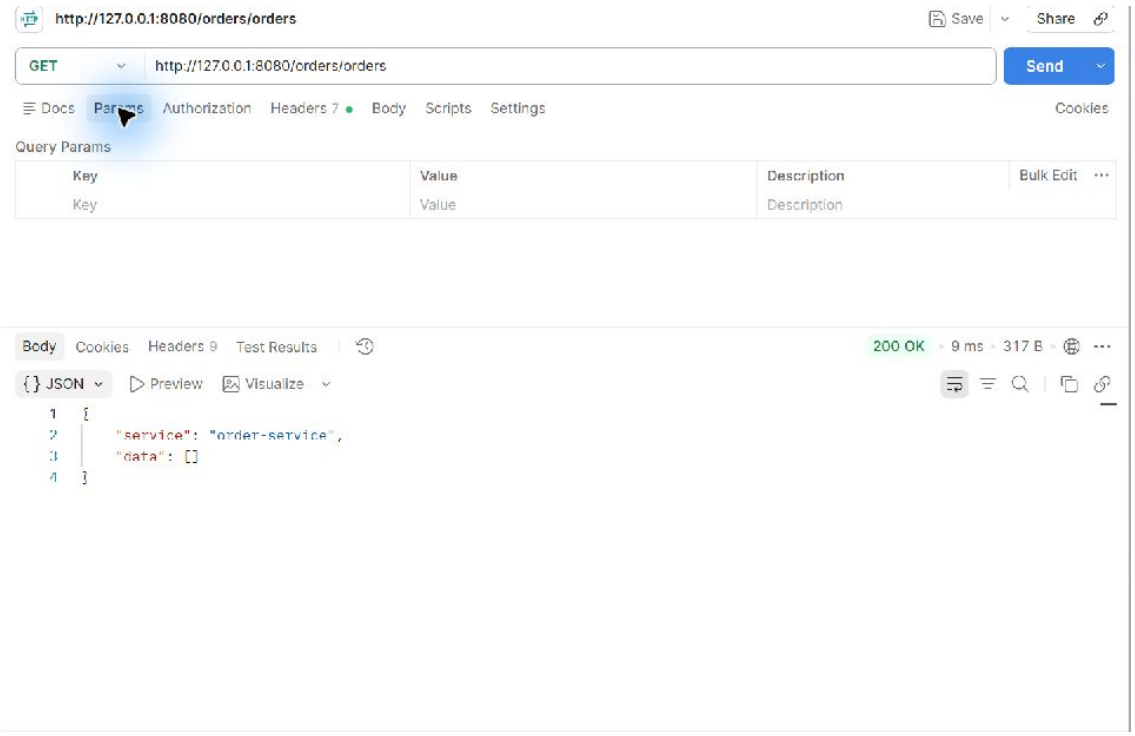
Jalankan `npm test` untuk suite otomatis. Untuk demonstrasi rate limit, hentikan stack terlebih dahulu lalu jalankan `npm run test:rate-limit`. Skrip membuat gateway baru di port 8080 dan mengirim 30 request berurutan. Jika menguji server yang sudah aktif dengan `node gateway/test-rate-limit.js`, tunggu setidaknya 10 detik tanpa request sebelumnya dari IP yang sama.

4. Bukti akses kedua service melalui gateway

Kedua tangkapan layar menunjukkan port gateway yang sama, yaitu 8080. Tampilan dipotong pada tata letak dokumen untuk memperbesar area request dan respons; berkas gambar asli tetap tersedia dalam repository.

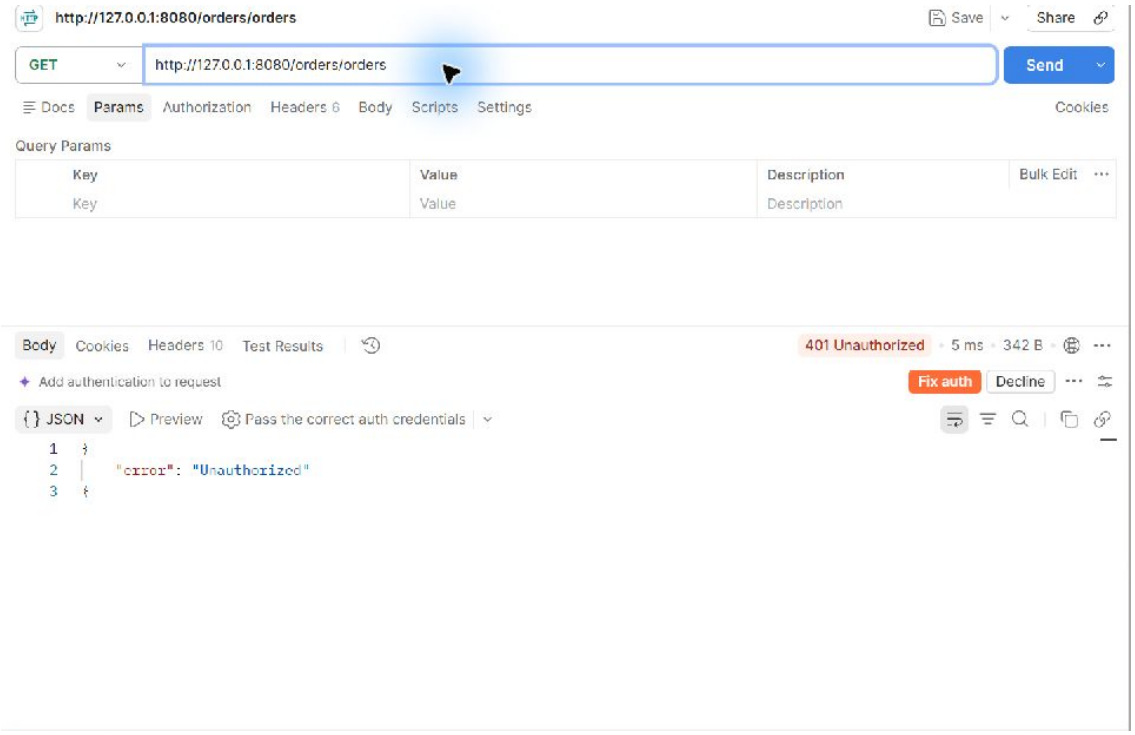


Gambar 2. Postman: `GET /catalog/products?search=kopi` melalui gateway menghasilkan 200 dan data produk sesuai pencarian.



Gambar 3. Postman: `GET /orders/orders` dengan key benar menghasilkan 200, `service: order-service`, dan daftar `data: []`. Nilai key tidak ditampilkan.

5. Verifikasi autentikasi dan pengujian otomatis



Gambar 4. Postman: request pesanan tanpa header `X-API-Key` ditolak gateway dengan `401 Unauthorized`.

5.1 Hasil suite otomatis

Eksekusi awal `npm test` menghasilkan **35 test lulus dari 35 test**, tanpa kegagalan, sesuai log `docs/evidence/tests.txt` berikut. Setelah perubahan deployment, suite menjadi 40 test; hasil terbarunya tercantum pada Bagian 8.

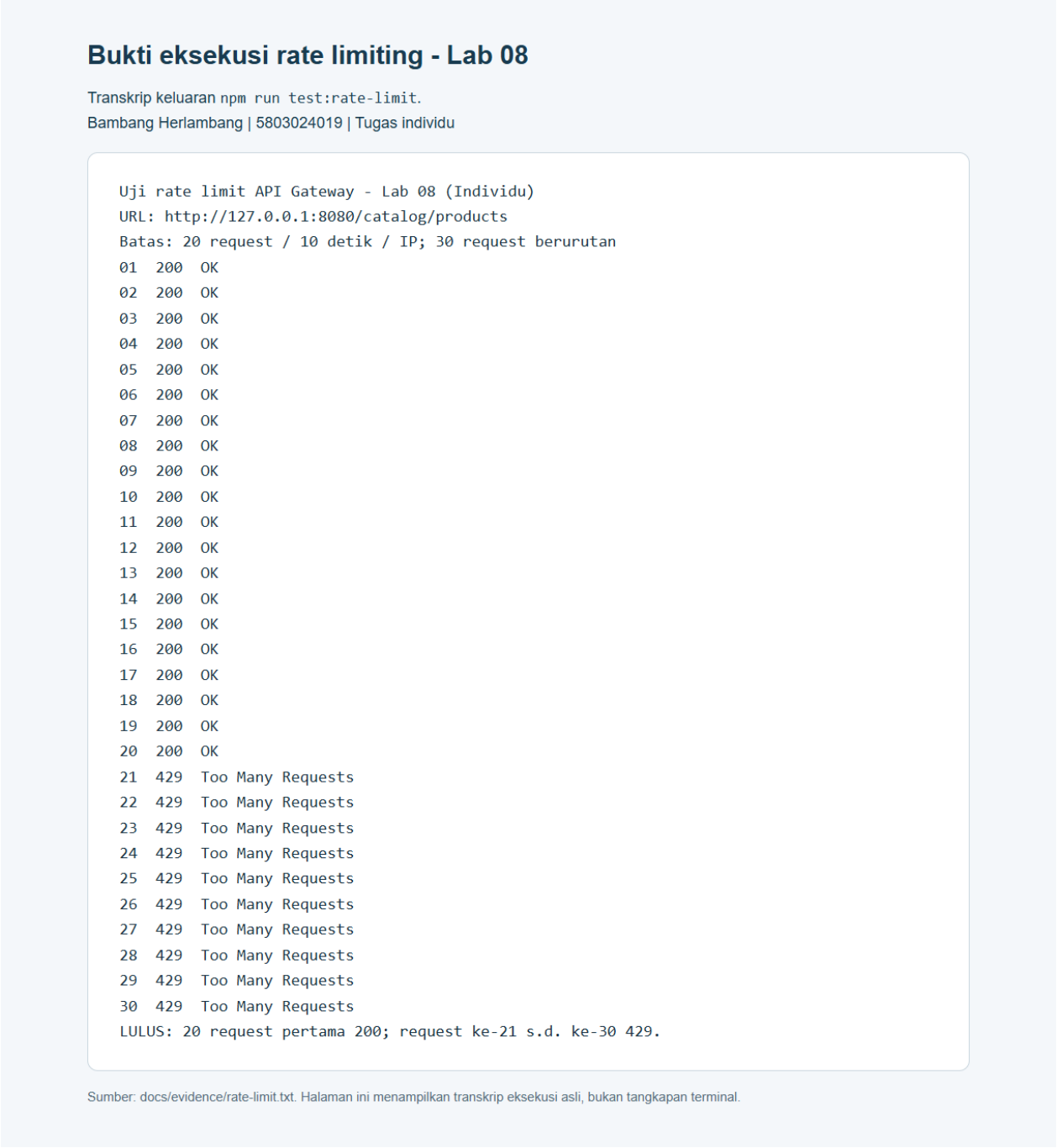
```
1..35
# tests 35
# pass 35
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 2805.1146
```

Pengujian alur nyata memeriksa katalog dan GraphQL melalui gateway, penolakan tanpa key, pembuatan pesanan `prod-001` sebanyak dua unit, total Rp170.000 dari harga katalog, pengambilan detail melalui header `Location`, serta daftar pesanan. Daftar kosong pada Gambar 3 berasal dari proses demo yang baru dijalankan; bukti pembuatan pesanan terdapat dalam suite otomatis.

Log memuat warning deprecation `DEP0060` terkait `util._extend` pada dependency proxy. Warning tersebut tidak menggagalkan assertion; hasil ini tidak dinyatakan sebagai eksekusi tanpa warning.

6. Bukti rate limiting

Demonstrasi memakai batas default **20 request per 10 detik per IP**. Skrip mengirim 30 request ke `http://127.0.0.1:8080/catalog/products` dengan kuota awal yang bersih.



Bukti eksekusi rate limiting - Lab 08

Transkrip keluaran npm run test:rate-limit.

Bambang Herlambang | 5803024019 | Tugas individu

```
Uji rate limit API Gateway - Lab 08 (Individu)
URL: http://127.0.0.1:8080/catalog/products
Batas: 20 request / 10 detik / IP; 30 request berurutan
01  200  OK
02  200  OK
03  200  OK
04  200  OK
05  200  OK
06  200  OK
07  200  OK
08  200  OK
09  200  OK
10  200  OK
11  200  OK
12  200  OK
13  200  OK
14  200  OK
15  200  OK
16  200  OK
17  200  OK
18  200  OK
19  200  OK
20  200  OK
21  429  Too Many Requests
22  429  Too Many Requests
23  429  Too Many Requests
24  429  Too Many Requests
25  429  Too Many Requests
26  429  Too Many Requests
27  429  Too Many Requests
28  429  Too Many Requests
29  429  Too Many Requests
30  429  Too Many Requests
LULUS: 20 request pertama 200; request ke-21 s.d. ke-30 429.
```

Sumber: docs/evidence/rate-limit.txt. Halaman ini menampilkan transkrip eksekusi asli, bukan tangkapan terminal.

Gambar 5. Tangkapan halaman transkrip keluaran `npm run test:rate-limit`, bersumber dari log eksekusi asli `docs/evidence/rate-limit.txt`; bukan tangkapan terminal langsung.

Request 1–20 menghasilkan `200`; request 21–30 menghasilkan `429 Too Many Requests`. Assertion skrip menyatakan lulus dan daftar status juga disimpan pada `docs/evidence/rate-limit.json`. Pengujian pemulihan window pada suite otomatis menggunakan window yang dipersingkat untuk efisiensi; demonstrasi pada gambar ini memakai konfigurasi default 10 detik.

7. Pembahasan, refleksi, dan referensi

7.1 Pembahasan hasil dan batasan

Postman membuktikan routing melalui satu port dan pemeriksaan key pada pesanan. Suite otomatis melengkapinya dengan verifikasi body POST, GraphQL, validasi input, harga katalog, kegagalan upstream, serta batas rate limiting.

Batasan implementasi yang masih berlaku:

- Penghitung request berada pada memori satu proses dan direset saat restart; beberapa instance memerlukan shared store.
- Data demo tersimpan dalam memori. Stok belum direservasi atau dikurangi; konsistensi pesanan bersamaan belum dijamin.
- Rotasi, pencabutan, scope API key, serta autentikasi pengguna belum diterapkan.
- Bukti awal berjalan lokal; deployment publik dibahas pada Bagian 8. Payment, komunikasi asinkron, OAuth2, dan mTLS berada di luar ruang lingkup tugas ini.

Pada audit 21 Oktober 2026, `braces@3.0.3` pada rantai `http-proxy-middleware -> micromatch -> braces` masih tercatat pada advisory GHSA-vfj7-8cjw-p6xm tanpa versi perbaikan. Gateway menggunakan `pathFilter` berupa fungsi dengan prefix statis, sehingga jalur pencocokan glob tersebut tidak digunakan oleh routing ini. Temuan dependency tetap dicatat; hasil audit tidak dinyatakan bersih.

7.2 Refleksi

Klien cukup mengetahui satu alamat gateway untuk mengakses katalog dan pesanan. Perubahan alamat atau port service dapat dilakukan melalui konfigurasi gateway tanpa mengubah alamat yang dipakai klien. Penerapan rate limit dan pemeriksaan API key di pintu masuk membuat kebijakan akses lebih mudah dipantau secara konsisten. Jika gateway mati, kedua service tidak dapat diakses melalui alamat tersebut sehingga gateway menjadi titik kegagalan yang perlu ditangani pada pengembangan berikutnya.

7.3 Kaitan dengan materi

Materi Web Service sesi 8 menjadi acuan header API key dan penyimpanan secret di environment. Materi Software Testing sesi 8 memberi dasar BVA, assertion, jalur negatif, serta prinsip pengujian yang independen dan dapat diulang. Framework aktual laporan ini adalah `node:test`; tugas Java/JUnit dan rancangan keamanan proyek UAS merupakan penugasan terpisah.

7.4 Referensi dan lokasi berkas

1. Philipus Suryo Subandoro. *Praktikum Web Service Lab 08: API Gateway Sederhana.* Materi lokal: `materi/2prakwebservice/sesi8.txt`.
2. Philipus Suryo Subandoro. *Web Service Sesi 08: Keamanan Lanjutan dan Kick-off Proyek.* `materi/1webservice/sesi8.txt`.
3. Philipus Suryo Subandoro. *Software Testing Sesi 08: Unit Testing (JUnit).* `materi/3softwaretesting/sesi8.txt`.
4. Dokumentasi resmi *http-proxy-middleware*: `https://github.com/chimurai/http-proxy-middleware`.
5. Dokumentasi resmi *express-rate-limit*: `https://express-rate-limit.mintlify.app/reference/configuration`.
6. Repository kode, collection Postman, dan bukti pengujian (GitHub).

8. Deployment publik pada Render

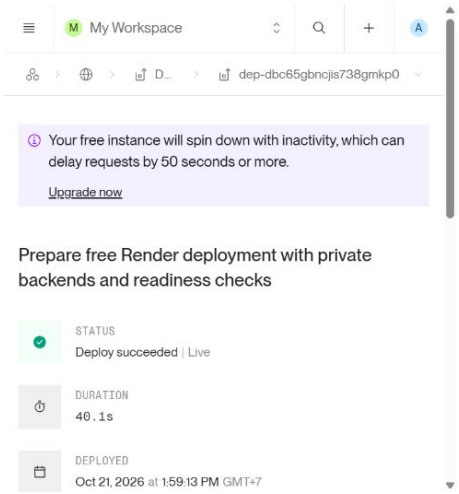
Gateway telah tersedia melalui HTTPS pada alamat berikut:

`https://prakwebservice-lab08-api-gateway.onrender.com`

Deployment pertama dari commit `196da4b` berhasil pada 21 Oktober 2026 pukul 13.59 WIB (GMT+7). Satu *Render Free Web Service* menjalankan `npm start` dengan tiga proses: gateway mendengarkan `0.0.0.0:PORT` (default Render `10000`), sedangkan backend hanya mendengarkan loopback pada port 3000 dan 3001. Ketiganya berada dalam satu instance layanan, bukan tiga instance terpisah.

Hasil verifikasi endpoint publik

Pemeriksaan	Hasil aktual
`GET /readyz`, awal dan setelah limit	`200` / `200`
Katalog GET / GraphQL POST	`200` / `200`
Pesanan tanpa / dengan key	`401` / `200`
Buat pesanan / detail `Location`	`201` / `200`
Kuantitas nol / route tidak dikenal	`400` / `404`
30 request pada window bersih	20×`200`, 10×`429`
Suite setelah perubahan deployment	40/40 lulus



Gambar 6. Status layanan *Live* pada dashboard Render dan ringkasan verifikasi publik. ID layanan: `srv-dbc65fbncjis738gmhlg`.

Pembuatan pesanan publik memakai `prod-001` sebanyak dua unit dan menghasilkan total Rp170.000. Detailnya berhasil dibaca melalui header `Location`. Pengujian kuota menunggu window bersih; readiness tetap `200` sesudah API menghasilkan `429`, karena `/readyz` tidak memakai kuota API.

Key deployment acak 256-bit disimpan lokal pada `gateway/.env.render`, yang diabaikan oleh Git. Klien memakai nilai `API_KEY` asli sebagai header `X-API-Key`; Render hanya menyimpan digest `API_KEY_SHA256`. Nilai key tidak dicantumkan dalam laporan atau bukti publik.

Layanan Free dapat tidur setelah 15 menit tanpa trafik; akses pertama dapat memerlukan sekitar satu menit untuk aktif kembali. Data pesanan berada dalam memori dan hilang ketika proses dimulai ulang. Hasil verifikasi publik ada pada `docs/evidence/render-verification.json` dan `docs/evidence/render-verification.txt`; hasil suite terbaru **40 test lulus, 0 gagal** ada pada `docs/evidence/tests-deployment.txt`.